AI-Generated Music Detection and its Challenges

Darius Afchar
Deezer Research

Gabriel Meseguer-Brocal
Deezer Research
Paris, France
research@deezer.com

Romain Hennequin
Deezer Research

arXiv:2501.10111v1 [cs.SD] 17 Jan 2025

***Abstract*—In the face of a new era of generative models, the detection of artificially generated content has become a matter of utmost importance. In particular, the ability to create credible minute-long synthetic music in a few seconds on user-friendly platforms poses a real threat of fraud on streaming services and unfair competition to human artists. This paper demonstrates the possibility (and surprising ease) of training classifiers on datasets comprising real audio and artificial reconstructions, achieving a convincing accuracy of 99.8%. To our knowledge, this marks the first publication of a AI-music detector, a tool that will help in the regulation of synthetic media. Nevertheless, informed by decades of literature on forgery detection in other fields, we stress that getting a good test score is not the end of the story. We expose and discuss several facets that could be problematic with such a deployed detector: robustness to audio manipulation, generalisation to unseen models. This second part acts as a position for future research steps in the field and a caveat to a flourishing market of artificial content checkers.**

***Index Terms*—music, generative ai, forgery, forensics**

I. Introduction

Generative models have gained tremendous popularity in the past couple of years. Many discussions have ensued around the new opportunities these models may provide, as well as critics about their sociotechnical context and the many risks they entail. It is tricky to talk about generative AI in a neutral manner, mainly because this trending topic has a larger reach than purely technical considerations (*e. g.,* commercial, legal, social and political ramifications [1]), and is fairly new for everyone. We also stress that so-called "neutral" discussions often support dominant views instead of truly enabling a scientific discourse encompassing all impacted stakeholders [2], [3]. As motivation for this work, we therefore explicitly call for better regulation of these models, for a number of reasons that interested readers may find discussed in detail in [4], [5], [6]. This constitutes a *working assumption* that we will not further debate in this paper.

One of the many areas that needs to be addressed in regulating AI-music is to better identify synthetic generations within a body of genuine human-made content. In 2023, a new wave of generative models has rendered the risks of *AI-generated music* more tangible than before [7], [8], [9], [10]. Several user-friendly services have also recently emerged and democratised the creation and diffusion of AI-music : *e. g.,* Riffusion[1], Suno[2], Udio [3], Stable Audio[4]. AI-music now pose a growing problem for music artists and labels. Lawsuits are currently being filed against several AI companies [1].

While studies have been conducted on detecting synthetic sounds and singing voices [11], [12], we present a novel setting in this paper. We propose the first general-purpose AI-generated music detector, a significant advancement that also includes generated instrumental parts. Our focus is on the trending waveform generators mentioned earlier. We leave symbolic or MIDI-based synthesis models for future exploration. Using basic convolutional models, we show that almost perfect detection scores (>99% accuracy) are easy to obtain.

Although AI-music detection is novel, we do not conduct our research in a vacuum. This task is very reminiscent of the topic of artificial forgery detection, within the field of media forensics: *e. g.,* deepfake detection, image tampering, voice spoofing [13], [11]. While these detectors are not directly transferable to the specifics of music, we can at least anticipate having to deal with similar research questions raised in this literature [14], [15]. Therefore, in the second part of our paper, we take a step back on our seemingly impressive results and discuss caveats to AI detectors: robustness to audio manipulation and generalisation to unseen generators. In a nutshell, *we have to look beyond performance scores*, no matter how good they look.

This paper serves as a first research study on AI-music detection and a proof of concept that they can be detected, but also as a position and message for the research community on the many facets and challenges to consider for the future research steps of this topic.

Our code is available at `github.com/deezer/deepfake-detector`.

II. AI-music detection

There are many ways to tackle AI-music detection. In this section, we first discuss our choice of framework and its advantage to solve the task, the employed data as well as some first surprisingly convincing detection scores.

A. Proposed framework

Motivated by the democratisation of online platforms that can generate minutes-long synthetic music, we restrict the scope of our paper to waveform based generators, common in these services: *e. g.,* MelGAN [16], HiFiGAN [17], Jukebox [18], Musika! [19], Moûsai [7], MusicLM [8], VampNet [9], MusicGen [10]. This list is not exhaustive of the swarm of published music generation methods. We only provide a representative subset.

While it is impossible to account for all particularities, we can usually break down these models into two parts. First, an autoencoder (AE) is trained to compress bits of raw audio into an easier representation to process and to invert this representation into an audio signal (*i. e.,* vocoder). For instance, mel-spectrogram representations were often used (*e. g.,* [16]) before being replaced by more recent so-called neural codecs – as Soundstream [20], Encodec [21] or DAC [22] – that demonstrated better reconstructions. For interested readers, the latter commonly employ discretised latent spaces as codebooks of tokens – *e. g., Residual Vector Quantization* (RVQ) [23]. Then, a second internal module is usually trained to learn to continue the compressed sequence temporally or generate it conditioned on text inputs, depending on the considered task. For instance, large-language-model (LLM) inspired architectures have been proposed for the role [9]. Put simply, the AE does the waveform synthesis

[1] `www.riffusion.com`
[2] `www.suno.ai`
[3] `www.udio.com`
[4] `stability.ai/stable-audio`

part while the LLM does the semantic work of generating a coherent musical sequence through time.

Detecting that a music sequence was artificially generated can be tricky. With the risk of falling into anthropomorphism, this equates to trying to learn a musician's style: *e. g.,* MusicGen might always generate music with a specific musical structure. Conversely, it might be easier to try to catch if an audio sample is the output of an AE. For instance, it is well-known that neural decoders tend to produce *checkerboard artefacts* [16] characteristic of transposed convolution operations. We might be able to catch many more such artefacts. Thus, we propose the following research direction: **can we detect if a music sample is generated by an artificial decoder, this independently of its musical content?**

Another difficulty is of a causal nature. If we collected real and synthetic music samples and naively trained a model to classify them, we might end up detecting features unrelated to generation artefacts. For instance, a public real music dataset might be full of classical music, while a synthetic dataset primarily includes rap and pop music. This could result in the classifier learning to detect classical music instead of distinguishing real and forgeries. This problem is known as *confounding* [24]. The same discussion applies to the compression codec that might confound the detection of AI-music (*e. g.,* all Riffusion songs are exported in mp3 192kbps).

These two remarks have motivated our following framework: We leverage a dataset of real music samples, which we auto-encode using the trained AE part of the above models. These samples are stored at the same bitrate as the original audio. Controlling on the music semantic and file encoding, the model we then train can only detect generation artefact since it should learn to tell apart a real audio from its reconstructed counterpart. Therefore, we limit these extraneous confounding influence [24], *i. e.,* shortcut learning.

B. Considered dataset and music generators

We chose to use the FMA dataset [25], an open dataset that allows reproducibility and comparison of future work. Due to size constraints, we only consider the medium split that includes 25.000 music tracks spread into 16 genres. All tracks are encoded in mp3 with a diversity of bitrates – with a majority of 320kbps, followed by 256 and 192kbps. The audio files are processed in 44.1kHz.

As for the autoencoders, we consider two popular neural codecs: Encodec (*e. g.,* used in *Suno's Bark* and MusicGen) and DAC (*e. g.,* used in Vampnet[5]). We also studied the decoder part of Musika, which was trained end-to-end on polar spectrograms. Finally, we consider a combination of a mel-spectrogram converter-inverter and a Griffin-Lim phase reconstruction (*e. g.,* used in *Riffusion v1*) – we dub this pipeline *GriffinMel*. Some audio reconstructions are available on our repository to gain intuition on each decoder. The availability of trained models constrained our choice of decoder. For instance, Soundstream is used as a latent representation in many of Google's models, yet no public checkpoint is available. The same applies to MelGAN and HiFiGan, for which no checkpoint exists for music data[6], as well as *Riffusion v3* and *Suno's Chirp* that are now closed-source. We consider several configurations for the above decoders: Encodec in 3, 6 and 24kbps, DAC in 2, 7 and 14kbps, and GriffinMel using 256 and 512 melbands.

We autoencode all considered real tracks and obtain nine different autoencoded reconstructions: *i. e.,* with the same "semantic" musical content and stored with the same bitrate, but with different artefacts linked to each AE. This leads to a dataset of 250.000 tracks. We split the songs (and their corresponding reconstructions) in a 70%, 10%, 20% fashion between train, validation, and test. Empirically, the GriffinMel reconstruction seems the easiest to catch due to audible phase errors. Encodec and DAC sound most challenging, especially at their maximum quality setting. If it is often possible to distinguish between a real and a reconstruction when placing one next to the other, it is way more tricky without a point of reference or being aware that the audio could be generated. This relates to the recent user study in [26].

[5]For interested readers, we actually use the LAC version of DAC that is better suited for music, similar to what is done in VampNet [9].

[6]Although there exists some for voice synthesis, we found that this resulted in heavy audible artefact on music that we deemed unrealistic.

TABLE I
TEST DETECTION SCORES FOR DIFFERENT AUDIO REPRESENTATIONS.

Model	Accuracy (%)
waveform	95.2
complex	92.9
amplitude	**99.8**
phase	99.6
polar	99.7

C. First results of detection

We started experimenting with our dataset with straightforward convolutional models (*i. e.,* alternating convolution and pooling layers). To our surprise, this basic setting led to test accuracies over 90%, which we were initially sceptical about. After several experiments, it seems this detection task is easier than we thought. As we will see next, our high scores did not prompt us to explore more complex models but rather to sanity check an already good performing model.

Briefly, our proposed model is composed of six convolutional layers with $[16, 32, 64, 128, 256, 512]$ filters. We use kernels of size 3 and a pooling of size 2. We finish with an average pooling and two linear layers. During training, real and synthetic music tracks are sampled with a $\frac{1}{2}$ probability. The synthetic one is sampled uniformly among the nine reconstructions. We extract a random 0.8s snippet from each track of the batch[7]. We use some common data preprocessing and augmentation of the audio: STFT, random mono mix, random gain, frequency cutoff at 16kHz[8], and conversion to decibel scale when applicable. All details and trained weights are available on our repository.

The architectural choices does not seem to impact the performance much. However, the choice of input preprocessing seems to be much more influential on our experiment results. We report in Table I the detection score for various choices of audio representations: the raw waveform, the complex STFT, its amplitude, its phase, or both stacked as polar coordinates. All our results are very satisfying overall. Transforming music samples as *amplitude* spectrograms leads to the best performance overall (**99.8%** accuracy). It is also interesting to see that the purely *phase*-based model yields high scores despite often being considered less efficient than the amplitude representation. A per-class breakdown is available in Table II and highlight consistent performances across different AE.

D. Generalisation to full music generators

We argued that instead of learning to detect AI-music generators, we could more simply learn to detect the fingerprint of the AE they employ. As a sanity check, we test our model on synthetic music created from text prompts, *i. e.,* unseen during training, not autoencoded

[7]Fixed arbitrarily and leading to 128 STFT time frames.

[8]... to avoid relying on spurious mp3 conversion artefacts.

TABLE II

ROBUSTNESS ACCURACY TEST SCORES. We test the accuracy of the amplitude-spectrogram-based model on various audio transforms (%). We include a breakdown per class. We report the previous *amplitude* model scores for comparison on top. Background colours highlight *score degradation*.

		All (%)	Encodec 3	Encodec 6	Encodec 24	GriffMel 256	GriffMel 512	DAC 2	DAC 7	DAC 14	Musika	Real
	base *amplitude* model	99.8	100.0	100.0	99.7	100.0	99.8	99.7	99.4	99.3	100.0	99.7
Audio manipulation	time stretch	88.6	76.3	76.4	72.5	83.4	72.2	84.4	79.1	79.2	76.5	99.4
	pitch shift	66.6	1.1	1.2	1.2	**99.3**	**97.5**	3.1	2.2	1.7	96.4	99.4
	EQ	**97.6**	97.4	97.4	95.1	98.0	96.6	93.9	91.7	91.8	**97.8**	99.7
	reverb.	96.9	**97.7**	**97.6**	**95.9**	94.1	91.2	**96.0**	**93.8**	**93.7**	87.2	99.7
	white noise	66.6	34.9	34.1	23.0	19.5	11.9	58.0	50.4	50.3	16.9	**99.9**
	mp3 64kB/s	73.8	57.0	27.2	7.0	92.4	41.3	69.7	44.3	48.6	49.4	99.3
	aac 64kB/s	58.4	12.7	4.5	0.8	56.9	26.7	7.5	2.9	3.3	40.7	99.7
	opus 64kB/s	64.6	8.9	13.0	15.3	90.1	55.0	8.7	8.8	8.3	63.4	99.5

nor related to FMA. We gather 50 tracks from MusicGen, amounting to 25 minutes of music, and extract random snippets from them. On a total of 2500 music snippets, we achieve a **99.9%** detection score.

We underscore that given our resulting performances, we did not find it so crucial to explore the best possible architecture further and deemed it more important to discuss the aftermath of obtaining such convincing scores. Indeed, it could feel that we have "solved" the task. Nevertheless, should we be so confident?

III. CAVEATS ON AI-MUSIC DETECTORS

Beyond the lab experiments, we find it crucial to ponder the consequence of deploying AI-music detectors to the world. Indeed, a new market of "AI content detectors" has emerged in recent years: *e. g.,* checking if a student essay employed ChatGPT[9]. Such tools often claim to have high detection scores. However, they are often closed-source, making verification tricky. This has notably led to strange situations where students have much trouble proving their good faith in false positive cases against the ethos of a so-called "trusted AI checker" [27]. Several commercial solutions have also recently been released for AI-music[10]. So far, they have followed the same path as AI-text detectors: closed-source and without any associated research publication. This does not allow rigorous studies [28].

This section hence discusses aspects to make AI-music detection more realistic and reliable. We discuss two main facets that make the detection more complex than may first appear: its robustness to audio manipulation and its generalisation to unknown encoders. We also highlight how untrustworthy performance numbers can be, which calls to make these detectors open source and considering other aspects to validate a model (*e. g.,* interpretability).

[9] *e. g.,* `gptzero.me`

[10] *e. g.,* `ircamamplify.io`, `pex.com`

A. Robustness to manipulations

An angle often discussed in the literature on forgery detection is the robustness to data shifts. There are countless scenarios where AI-music creators do not directly publish the immediate output of the generative model. For instance, they could genuinely reencode them in a different format while exporting the result or adding it to a video clip. They could also try to bypass a detector more strategically by applying time-stretching or pitch shift transforms, similar to what is frequently done on social networks to bypass fingerprint systems and evade copyright claims. It would be unrealistic not to expect some users to try to evade detection.

As a first study, we consider some common audio transformations that lay users could employ: random pitch shift (± 2 semitones), time stretch ($[80, 120]\%$), EQ, reverb, addition of white noise, reencoding in *mp3*, *aac*, and *opus* in 64kbps. Implementation details are available in our repository. We leave attacks from more advanced users for future work (*e. g.,* adversarial attacks [28]).

We evaluate the amplitude-based model from the previous section on such unseen transformation and report the results in Table II. The performances drop drastically under pitch shifts, the addition of white noise, and codec reencoding. This is consistent with previous literature on forgery detection that ML models are generally not robust to out-of-distribution shifts – if not explicitly designed for them [29], [30]. Conversely, it is unclear why the model remains robust to some manipulations (*i. e.,* time stretch, EQ and reverberation).

We highlight that several scores drop to almost zero, which means that the model has predicted the real class for most samples (instead of more unconfident, aleatoric guesses). Meanwhile, the manipulations did not impact the *real* class scores. This suggests that the model works by detecting artefacts specific to each AE and otherwise defaulting to the *real* class if none is found (or that the manipulations make them unrecognisable for the model). This would not be surprising since the *real* class can be expected to be more